

# QUANTUM COMPUTING FOR NON-PHYSICISTS: OVERVIEW

Yogesh Haribhau Kulkarni

# Outline

- 1 INTRODUCTION AND MOTIVATION
- 2 FROM BITS TO QUBITS
- 3 RULES OF THE QUANTUM WORLD
- 4 QUANTUM MACHINE LEARNING
- 5 REFERENCES

# About Me

# Yogesh Haribhau Kulkarni

## Bio:

- ▶ 20+ years in CAD/Engineering software development
- ▶ Bachelors, Masters and Doctoral degrees in Mechanical Engineering (specialization: Geometric Modeling Algorithms).
- ▶ Coaching in Data Science, AI/ML/DL, NLP, and Quantum Computing.
- ▶ Feel free to follow me at:
  - ▶ Github ([github.com/yogeshhk](https://github.com/yogeshhk))
  - ▶ LinkedIn ([www.linkedin.com/in/yogeshkulkarni/](https://www.linkedin.com/in/yogeshkulkarni/))
  - ▶ Medium ([yogeshharibhaukulkarni.medium.com](https://yogeshharibhaukulkarni.medium.com))
  - ▶ Send email to [yogeshkulkarni](mailto:yogeshkulkarni@yahoo.com) at yahoo dot com



Office Hours:  
Saturdays, 2 to 5pm  
(IST); Free-Open to all;  
email for appointment.

## About Content

- ▶ Quantum Technologies: Computing, Communication, Sensing, and beyond.
- ▶ Focused on foundations needed for practitioners and engineers.
- ▶ Each sub-field can go extremely deep – this is the essential starting point.
- ▶ Prior knowledge of linear algebra and complex numbers is helpful but not required.

## About Me

- ▶ I am not a physicist.
- ▶ And this course will not turn YOU into one either!
- ▶ But it will give you solid foundations to explore further.
- ▶ Goal: make quantum concepts accessible to engineers and programmers.

# Quantum Computing for Non-Physicists

## Module 0: Introduction and Motivation

## What This Course Is (and Is Not)

- ▶ This is NOT a physics course — no wave functions, no Schrödinger equation
- ▶ This IS a programmer's guide to quantum computing
- ▶ You will understand what quantum computers do and why it matters
- ▶ You will write and run real quantum circuits using Qiskit
- ▶ Assumed background: school-level physics, programming in Python



# Why Should Programmers Care?

- ▶ Classical computers are hitting physical limits (transistors approaching atom size)
- ▶ Quantum computers solve certain problems exponentially faster
- ▶ Drug discovery: simulating molecules is intractable classically for large molecules
- ▶ Cryptography: current encryption will be broken by sufficiently large quantum computers
- ▶ Optimization: logistics, finance, machine learning training
- ▶ Industry is investing billions — IBM, Google, Microsoft, Amazon, IonQ

# The Classical Computing Wall

- ▶ Moore's Law: transistor count doubles every 2 years (since 1965)
- ▶ Transistors are now just a few atoms wide — quantum effects become a problem
- ▶ Clock speed has stalled around 3–4 GHz since 2004
- ▶ More cores help, but some problems cannot be parallelized classically
- ▶ Energy cost of computation is rising unsustainably

# NP-Hard Problems

- ▶ Problems where complexity grows exponentially
- ▶ Each additional variable dramatically increases computation
- ▶ Example: 1 variable = 2 possibilities, 10 variables = 1024 possibilities
- ▶ 100 variables = astronomically large search space
- ▶ Creates combinatorial explosion
- ▶ Resource requirements grow as  $2^n$
- ▶ Classical computers eventually run out of memory and time

# Why Quantum Helps

- ▶ Exploits superposition and quantum state combinations
- ▶ Parallel probability exploration
- ▶ Some exponential problems behave more like linear
- ▶ Instead of checking one state at a time, quantum systems represent multiple states
- ▶ Massive search advantages
- ▶ Optimization advantages
- ▶ Potential exponential speedups for certain problems

## What Makes Quantum Different?

- ▶ Quantum computers exploit quantum mechanical properties of matter
- ▶ A quantum bit (qubit) can be 0, 1, or both at the same time (superposition)
- ▶ Multiple qubits can be linked in ways classical bits cannot (entanglement)
- ▶ These properties let quantum computers explore many solutions simultaneously
- ▶ Not faster at everything — quantum advantage exists for specific problem types

# Classical vs Quantum Computing

- ▶ Classical computers excel at: spreadsheets, web browsing, standard software
- ▶ Handle linear or polynomial complexity problems well
- ▶ Linear scaling: 10x workload requires roughly 10x resources
- ▶ Quantum computers are NOT replacements for classical
- ▶ They are specialized accelerators for specific problem classes
- ▶ Analogy: GPUs did not replace CPUs, they complement them
- ▶ Quantum computers will coexist with classical systems

# Key Applications of Quantum Computing

- ▶ **Drug Discovery:** Simulate molecular interactions at quantum level
- ▶ **Cryptography:** Break RSA encryption; quantum-safe replacements
- ▶ **Optimization:** Portfolio optimization, logistics routing, scheduling
- ▶ **Machine Learning:** Faster training, quantum kernels, quantum neural networks
- ▶ **Material Science:** Design new superconductors, batteries, catalysts
- ▶ **Weather/Climate:** High-fidelity climate simulations

# Shor's Algorithm & Encryption

- ▶ One of the most famous quantum algorithms
- ▶ Modern encryption depends on prime factorization difficulty
- ▶ RSA encryption:  $N = p \times q$  (two large primes)
- ▶ Easy to multiply primes, extremely hard to factor
- ▶ Classical computers struggle with factorization
- ▶ Shor's algorithm: quantum computers can factor quickly
- ▶ Implication: current encryption systems could break
- ▶ Post-quantum cryptography becoming important



# A Brief History of Quantum Computing

- ▶ 1980: Feynman proposes using quantum systems to simulate quantum physics
- ▶ 1994: Shor's algorithm — quantum factoring threatens RSA encryption
- ▶ 1996: Grover's algorithm — quadratic speedup for search
- ▶ 2001: IBM demonstrates 7-qubit Shor's algorithm
- ▶ 2016: IBM puts a 5-qubit quantum computer on the cloud (IBM Quantum Experience)
- ▶ 2019: Google claims “quantum supremacy” with 53-qubit Sycamore processor
- ▶ 2023+: IBM 1000+ qubit processors; IonQ, Quantinuum, PsiQuantum scale up

# The Quantum Computing Landscape Today

- ▶ **Superconducting:** IBM (Eagle, Osprey, Heron), Google (Sycamore), Rigetti
- ▶ **Trapped Ion:** IonQ, Honeywell/Quantinuum
- ▶ **Photonic:** PsiQuantum, Xanadu
- ▶ **Neutral Atom:** Pasqal, QuEra, Atom Computing
- ▶ **Silicon Spin:** Intel, Silicon Quantum Computing
- ▶ **Cloud access:** IBM Quantum, AWS Braket, Azure Quantum, Google Cloud Quantum AI

## NISQ Era: Where We Are Now

- ▶ NISQ = Noisy Intermediate-Scale Quantum
- ▶ Today's machines: 50–1000+ physical qubits, but noisy (error rates  $\sim 0.1\text{--}1\%$ )
- ▶ Not yet fault-tolerant — computations are limited in depth
- ▶ Still useful for certain problems: chemistry simulation, optimization, ML
- ▶ Fault-tolerant quantum computing (FTQC) is the long-term goal
- ▶ FTQC may need millions of physical qubits for useful logical qubits

# The Quantum Software Stack

- ▶ **Application layer:** Algorithms (Grover, Shor, VQE, QAOA)
- ▶ **Framework layer:** Qiskit (IBM), Cirq (Google), PennyLane, Braket (Amazon)
- ▶ **Compiler/transpiler:** Converts circuits to native gate sets
- ▶ **Hardware layer:** Actual qubits (superconducting, trapped ion, etc.)
- ▶ As a programmer, you mostly work at the framework and algorithm layer

# How to Think About Quantum Computing

- ▶ Think of qubits as *probabilistic* bits, not just 0 or 1
- ▶ Quantum algorithms set up probability amplitudes so right answers are likely
- ▶ Measurement “collapses” the quantum state — run circuits many times
- ▶ Intuition, not equations, is your primary tool as a programmer
- ▶ Classical analogies help initially but will break down — embrace the weirdness

# Course Roadmap

- ▶ **Workshop A:** Bits to Qubits, Rules of Quantum World (Foundations)
- ▶ **Workshop B:** Qubit Gates, Circuits, Qiskit Programming
- ▶ **Workshop C:** Quantum Algorithms (Grover, QFT, Shor's, VQE, QAOA)
- ▶ **Workshop D:** Hardware, Communication, Information, Simulations, Applications
- ▶ Throughout: Hands-on Qiskit labs after every major topic
- ▶ Capstone: Mini-project on a quantum computing problem of your choice

## Tools and Setup

- ▶ **Python 3.8+** and **Jupyter Notebook** (Anaconda recommended)
- ▶ **IBM Quantum account**: Free at [quantum.ibm.com](https://quantum.ibm.com) — needed for real hardware

```
1 # Install Qiskit and simulator backend
  pip install qiskit qiskit-aer qiskit-ibm-runtime
3
  # Optional: PennyLane for QML labs
5 pip install pennylane
7
  # Verify installation
  python -c "import qiskit; print(qiskit.version.get_version_info())"
```

# What Quantum Computers Cannot Do

- ▶ NOT a general-purpose speedup machine — quantum advantage requires exploitable structure
- ▶ NOT a replacement for classical computers — works alongside as a specialized coprocessor
- ▶ NOT faster for everyday tasks: web browsing, video, databases, spreadsheets, Zoom calls
- ▶ Analogy: GPUs did not replace CPUs, they specialize — quantum will do the same
- ▶ Quantum advantage is problem-specific: search, factoring, molecular simulation
- ▶ Understanding limitations is as important as understanding capabilities



## Exercise: Quantum Use Case Quiz

- ▶ Which of these problems might benefit from quantum computing?  
(discuss in pairs)
  1. Finding a name in a sorted phone book
  2. Factoring a 2048-bit number used in RSA encryption
  3. Simulating the folding of a protein with 50 amino acids
  4. Sorting 1 million integers
  5. Finding the optimal route for 100 delivery vehicles
  6. Training a deep neural network on ImageNet
- ▶ Answers and reasoning: next slide

## Exercise: Quantum Use Case Quiz (Answers)

- ▶ (1) Sorted search: classical binary search is already  $O(\log n)$  — no quantum advantage
- ▶ (2) RSA factoring: YES — Shor's algorithm gives exponential speedup
- ▶ (3) Protein folding: YES — quantum simulation is exponentially more efficient
- ▶ (4) Sorting: No clear advantage (Grover gives at best  $O(\sqrt{n})$  comparisons)
- ▶ (5) Vehicle routing: YES — QAOA can tackle combinatorial optimization
- ▶ (6) Neural network training: Unclear; quantum ML is active research, not proven yet

## Important Terms

- ▶ Qubit: quantum version of a bit
- ▶ Superposition: ability to exist in multiple states simultaneously
- ▶ NP-hard: problems with exponential complexity
- ▶ Optimization: finding best solution among many possibilities
- ▶ QUBO: Quadratic Unconstrained Binary Optimization format
- ▶ Ising Model: physics-based optimization framework
- ▶ Shor's Algorithm: quantum factorization algorithm
- ▶ Photonic Quantum Computing: quantum systems using photons/light

## Reading List and Resources

- ▶ **Books:** “Quantum Computing: An Applied Approach” by Jack Hidary; “Programming Quantum Computers” by Johnston et al.
- ▶ **Online:** IBM Quantum Learning ([learning.quantum.ibm.com](https://learning.quantum.ibm.com)); Qiskit Textbook ([qiskit.org/learn](https://qiskit.org/learn))
- ▶ **Courses:** MIT 8.370x; Berkeley CS 294; Coursera “Quantum Computing from Basics to Cutting Edge”
- ▶ **YouTube:** Qiskit channel, Quantum Sense, 3Blue1Brown (linear algebra)
- ▶ **Community:** Qiskit Slack, Quantum Computing Stack Exchange

# Module 1: From Bits to Qubits

# Classical Bits: The Building Block of Computing

- ▶ Every classical computer stores information as bits: 0 or 1
- ▶ Physically realized as: voltage levels (low/high), magnetic domains, optical pulses
- ▶ All classical computation: manipulating sequences of 0s and 1s
- ▶ A byte = 8 bits, represents 256 possible values
- ▶ An  $n$ -bit register holds exactly ONE value at a time out of  $2^n$  possibilities

# From Bits to Qubits

- ▶ A **classical bit** is exactly 0 or 1 — deterministic, readable without disturbing it
- ▶ A **qubit** is the quantum analogue: when measured it gives 0 or 1
- ▶ Unlike a classical bit, before measurement a qubit can be in *superposition* of both
- ▶ Qubit state lives in a 2-dimensional complex vector space
- ▶ Physical realizations: electron spin, photon polarization, superconducting circuit energy level
- ▶ The measurement outcome is probabilistic, not deterministic
- ▶ For  $n$  qubits:  $2^n$  possible states can be represented simultaneously

# Quantum State of a Qubit

- ▶ State is a vector of unit length in 2D complex vector space
- ▶ For example:  $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$  or  $\begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix}$
- ▶ General form:  $\alpha |0\rangle + \beta |1\rangle$  where  $\alpha, \beta$  are complex numbers
- ▶ **Ket notation**  $|\cdot\rangle$  (Dirac bra-ket):  $|0\rangle$  and  $|1\rangle$  represent basis states
- ▶ Contains all information about the quantum system



# Superposition: The Coin Analogy

- ▶ Imagine a coin:
  - ▶ Classical: coin flat on table — heads (0) or tails (1), definite state
  - ▶ Quantum: coin spinning in the air — neither heads nor tails until it lands
- ▶ A qubit in superposition has a *probability amplitude* for 0 and for 1
- ▶ When we measure, it “lands” on 0 or 1 with probabilities set by the amplitudes
- ▶ The probabilities must sum to 1 (certainty)
- ▶ Key: the probabilities are determined before measurement but the outcome is random

# Superposition Explained

- ▶ Qubit exists in multiple states at once until measured
- ▶ Calculation processes all superposed values together
- ▶ Output calculated for all continuous values simultaneously
- ▶ Gate operations manipulate probability amplitudes
- ▶ Clever gates increase probability of desired outcome
- ▶ When measured, most likely to collapse to solution
- ▶ Enables massive parallelism in computation

## Probability Amplitudes (No Heavy Math)

- ▶ A qubit state is described by two numbers  $\alpha$  and  $\beta$  (complex amplitudes)
- ▶ The probability of measuring 0 is  $|\alpha|^2$ ; probability of measuring 1 is  $|\beta|^2$
- ▶ They must satisfy:  $|\alpha|^2 + |\beta|^2 = 1$  (probabilities sum to 1)
- ▶ Example:  $\alpha = \beta = 1/\sqrt{2}$  gives 50% chance of 0, 50% chance of 1
- ▶ This is the *uniform superposition* state (like a fair coin)

# The Bloch Sphere: Visualizing a Qubit

- ▶ Every possible state of a qubit maps to a point on the surface of a sphere
- ▶ North pole = state  $|0\rangle$  (classical 0)
- ▶ South pole = state  $|1\rangle$  (classical 1)
- ▶ Equator = equal superposition (50/50 chance of 0 or 1)
- ▶ Any point in between = some superposition
- ▶ The angle from the north pole determines the probability ratio
- ▶ The “longitude” angle encodes the relative phase (important for interference)

## Quantum Measurement: Collapse

- ▶ Measuring a qubit in superposition gives a random 0 or 1 outcome
- ▶ State  $\alpha|0\rangle + \beta|1\rangle$ : outcome 0 with probability  $|\alpha|^2$ , outcome 1 with  $|\beta|^2$
- ▶ After measurement: the qubit *collapses* to a definite state — superposition is destroyed
- ▶ Post-measurement state:  $|0\rangle$  if outcome was 0,  $|1\rangle$  if outcome was 1
- ▶ Measurement is **irreversible** — quantum gates are reversible; measurement is not
- ▶ Always measure at the end of a quantum circuit, not in the middle
- ▶ Run circuits many times to build a probability distribution

# Measurement Probabilities

- ▶ Normalization:  $|\alpha|^2 + |\beta|^2 = 1$  (probabilities always sum to 1)
- ▶ Circuit notation: a semicircle (meter) symbol with a double wire output denotes measurement
- ▶ The double wire carries a classical bit  $m \in \{0, 1\}$  — the result we read out
- ▶ After measurement, all quantum information is collapsed to classical
- ▶ This is why we run circuits many times: to estimate the probability distribution

## Why Can't We Just Look at a Qubit?

- ▶ Observation disturbs quantum systems — this is fundamental, not a technical limitation
- ▶ You cannot check a qubit's state without collapsing it
- ▶ This is related to the Heisenberg Uncertainty Principle
- ▶ This has practical consequences for quantum algorithms:
  - ▶ We cannot read out partial results mid-computation
  - ▶ We must design algorithms so the right answer has high probability at the end
- ▶ Quantum debugging is harder than classical debugging!

## Multiple Qubits: Exponential State Space

- ▶ 1 qubit: 2 possible states (0 or 1)
- ▶ 2 qubits: 4 possible states (00, 01, 10, 11)
- ▶ 3 qubits: 8 possible states
- ▶  $n$  qubits:  $2^n$  possible states
- ▶ A quantum computer with  $n$  qubits can be in a superposition of ALL  $2^n$  states
- ▶ 50 qubits:  $2^{50} \approx 10^{15}$  states simultaneously
- ▶ 300 qubits: more states than atoms in the observable universe



## Quantum Parallelism: The Key Advantage

- ▶ A quantum computer can apply an operation to ALL  $2^n$  states simultaneously
- ▶ This is not the same as running  $2^n$  parallel classical computers
- ▶ The trick: we can only extract  $n$  bits of information on measurement
- ▶ Quantum algorithms must cleverly funnel amplitudes towards the right answer
- ▶ The art of quantum algorithms = interference engineering
- ▶ Constructive interference: amplify the correct answer
- ▶ Destructive interference: suppress the wrong answers

## Comparing Classical and Quantum Bits

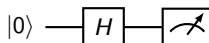
- ▶ **Classical bit:** 0 or 1, deterministic, cheap to copy, read without disturbance
- ▶ **Qubit:** probabilistic mixture before measurement, collapses on reading, cannot be copied (no-cloning)
- ▶ Both collapse to 0 or 1 when read — the difference is what happens in between
- ▶ Quantum states live in a much larger mathematical space (complex amplitudes)
- ▶ This extra “richness” is the source of quantum computational power

## Physical Realizations: What IS a Qubit?

- ▶ **Electron spin:** spin up =  $|0\rangle$ , spin down =  $|1\rangle$
- ▶ **Photon polarization:** horizontal =  $|0\rangle$ , vertical =  $|1\rangle$
- ▶ **Superconducting circuit:** ground energy state =  $|0\rangle$ , excited =  $|1\rangle$
- ▶ **Trapped ion:** two energy levels of an ion atom
- ▶ All of these behave as two-level quantum systems
- ▶ The abstraction (qubit) is the same regardless of physical implementation

## Lab Preview: Your First Qubit in Qiskit

- ▶ Create a qubit in  $|0\rangle$ , apply Hadamard gate  $H$ , measure 1000 times
- ▶ Expected:  $\approx 500$  zeros and 500 ones — demonstrating equal superposition



```
from qiskit import QuantumCircuit
2 from qiskit_aer import AerSimulator
qc = QuantumCircuit(1, 1)
4 qc.h(0)          # Hadamard: create superposition
  qc.measure(0, 0) # Measure qubit 0 into classical bit 0
```

## Lab: Bloch Sphere Visualization

- ▶ Visualize qubit states  $|0\rangle$ ,  $|1\rangle$ ,  $|+\rangle$ ,  $|-\rangle$  on the Bloch sphere
- ▶ Builds geometric intuition for quantum states before moving to gates

```
1 from qiskit.visualization import plot_bloch_multivector
  from qiskit.quantum_info import Statevector
3 sv = Statevector.from_label('++')
  plot_bloch_multivector(sv)
```

## Qubit States: Key Reference Points

- ▶  $|0\rangle$  — north pole, always measures 0
- ▶  $|1\rangle$  — south pole, always measures 1
- ▶  $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$  — equator, 50/50 in Z basis
- ▶  $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$  — equator, 50/50 but opposite phase
- ▶  $|i\rangle = (|0\rangle + i|1\rangle)/\sqrt{2}$  — equator, Y axis
- ▶ The phase difference between  $|+\rangle$  and  $|-\rangle$  matters for interference

# Computational Basis States

- ▶ Two special states correspond to classical 0 and 1
- ▶  $|0\rangle := \begin{bmatrix} 1 \\ 0 \end{bmatrix}$  (computational basis state for 0)
- ▶  $|1\rangle := \begin{bmatrix} 0 \\ 1 \end{bmatrix}$  (computational basis state for 1)
- ▶ These are orthonormal vectors
- ▶ Serve as foundation for general quantum states
- ▶ Any qubit state is a linear combination of these

## General States of a Qubit

- ▶ Beyond  $|0\rangle$  and  $|1\rangle$ , many more states are possible
- ▶ Example:  $0.6|0\rangle + 0.8|1\rangle$
- ▶ Superposition: linear combination of basis states
- ▶ Amplitude: coefficient for a particular state
- ▶ In general:  $\alpha|0\rangle + \beta|1\rangle$  where  $\alpha, \beta \in \mathbb{C}$
- ▶ These extra states give quantum computers power beyond classical



# Normalization Constraint

- ▶ Sum of squares of amplitudes must equal 1
- ▶ For state  $\alpha |0\rangle + \beta |1\rangle$ :  $|\alpha|^2 + |\beta|^2 = 1$
- ▶ Example:  $(0.6)^2 + (0.8)^2 = 0.36 + 0.64 = 1 \checkmark$
- ▶ Ensures quantum state is a unit vector
- ▶ Related to measurement probabilities summing to 1
- ▶ Fundamental requirement for valid quantum states

## Quantum States: The Water Glass Analogy

- ▶ Imagine filling a glass with colored water as a metaphor for probability amplitudes
- ▶ The fraction of water in the 0-half = probability of measuring 0
- ▶ You can tilt the glass (rotate the state) without spilling (unitary operation)
- ▶ When you set the glass down flat (measure), it snaps to one side
- ▶ The snap is random but weighted by how much water was on each side
- ▶ Gates = controlled tilting; measurement = letting it settle

## From One Qubit to Quantum Registers

- ▶ A quantum register is  $n$  qubits working together
- ▶ The state of  $n$  qubits lives in a  $2^n$ -dimensional space
- ▶ `Qiskit QuantumCircuit(n)` creates an  $n$ -qubit register
- ▶ Qubits in a register can be independent or entangled (correlated)
- ▶ Entanglement is the key to quantum advantage over independent qubits
- ▶ We will explore entanglement in Module 2

# Why Classical Simulation of Qubits is Exponentially Hard

- ▶ To simulate  $n$  qubits classically, you need  $2^n$  complex numbers
- ▶ 50 qubits:  $2^{50} = 1$  quadrillion amplitudes  $= \sim 8$  petabytes of memory
- ▶ This is why quantum computers are hard to simulate and why they matter
- ▶ Current classical supercomputers can simulate up to  $\sim 50$ – $55$  qubits
- ▶ Beyond that, only a quantum computer can efficiently handle the state
- ▶ This is the core of “quantum computational advantage”

## Exercise: Qubit Probability Calculations

- ▶ A qubit has amplitudes  $\alpha = \sqrt{3}/2$  for  $|0\rangle$  and  $\beta = 1/2$  for  $|1\rangle$ 
  1. What is the probability of measuring 0? (Answer:  $3/4$ )
  2. What is the probability of measuring 1? (Answer:  $1/4$ )
  3. Do they sum to 1? Verify:  $(3/4) + (1/4) = 1$
  4. After measuring 0, what is the qubit's state? (Answer:  $|0\rangle$ , collapsed)
- ▶ **Extended:** Write a Qiskit circuit that produces these probabilities using the Ry gate

## Module 1 Summary

- ▶ Classical bits are definite (0 or 1); qubits can be in superposition
- ▶ Superposition = probability amplitudes for both 0 and 1 simultaneously
- ▶ Measurement collapses the qubit to a definite value ( $|0\rangle$  or  $|1\rangle$ ), destroying superposition
- ▶ The Bloch sphere is the standard visual for a single qubit state
- ▶  $n$  qubits span  $2^n$  states simultaneously, enabling quantum parallelism
- ▶ Quantum advantage comes from interference: amplifying right, suppressing wrong
- ▶ Next: The fundamental rules that govern quantum systems

## Module 2: Rules of the Quantum World

# The Quantum Rule Book

- ▶ Quantum mechanics has a small set of fundamental rules
- ▶ These rules are counterintuitive but extremely well-tested experimentally
- ▶ We will cover 6 rules that matter most for quantum computing:
  1. Superposition
  2. Measurement and Collapse
  3. Entanglement
  4. Interference
  5. No-Cloning
  6. Decoherence
- ▶ Think of these as the “physics laws” your programs must respect



## Rule 1: Superposition

- ▶ A quantum system can exist in multiple states simultaneously
- ▶ Only takes a definite value when measured
- ▶ Not just a lack of knowledge about the state — the system genuinely has no definite value
- ▶ Demonstrated by the double-slit experiment: a single particle goes through both slits
- ▶ For qubits: can be in both 0 and 1 at the same time
- ▶ Enables quantum parallelism: apply operations to all states simultaneously

## The Double-Slit Experiment (Intuition)

- ▶ Shine light (or electrons) through two slits onto a screen
- ▶ Classical particle: goes through one slit, makes two bands on screen
- ▶ Quantum particle: behaves like a wave, goes through both slits simultaneously
- ▶ Result: interference pattern (alternating bright and dark bands)
- ▶ If you try to detect which slit the particle went through: interference disappears!
- ▶ Observation itself changes the quantum system
- ▶ Moral: quantum objects don't have a definite path until observed

## Rule 2: Measurement and Collapse

- ▶ Measuring a qubit in superposition gives a random 0 or 1 outcome
- ▶ The probability of each outcome is set by the amplitudes
- ▶ After measurement: the qubit is in a definite state (collapsed)
- ▶ Measurement is *irreversible* — you cannot undo it
- ▶ Quantum gates (before measurement) are reversible; measurement is not
- ▶ Always measure at the end of a quantum circuit, not in the middle

## Rule 3: Entanglement

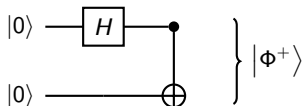
- ▶ Two or more qubits can be entangled: their states are correlated
- ▶ Measuring one entangled qubit instantly determines the other's measurement outcome
- ▶ This correlation holds regardless of distance between the qubits
- ▶ Einstein called it “spooky action at a distance” — he didn't like it
- ▶ Bell's experiments (1980s+) confirmed entanglement is real
- ▶ Entanglement is not telepathy: you cannot use it to send information faster than light

## Entanglement: The Glove Analogy

- ▶ Imagine you put one glove in each of two boxes, seal them, and send them to different cities
- ▶ When one person opens their box and finds a left glove, they know the other has the right one
- ▶ Classical: the gloves had definite left/right identities when packed — no mystery
- ▶ Quantum: the gloves had NO definite identities until one was opened — truly random until then
- ▶ Bell's theorem and experiments rule out the classical (pre-assigned) explanation
- ▶ Entanglement is a real non-classical correlation

## Creating Entanglement: Bell States

- ▶ The simplest entangled state:  $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$
- ▶ Measure qubit A = 0  $\Rightarrow$  qubit B guaranteed 0; same for 1 — perfect correlation
- ▶ Circuit: Hadamard on qubit 0, then CNOT (qubit 0 controls qubit 1)



```
qc = QuantumCircuit(2, 2)
2 qc.h(0)           # Superposition on qubit 0
  qc.cx(0, 1)       # CNOT: entangle qubit 0 (ctrl) with qubit 1 (tgt)
4 qc.measure([0, 1], [0, 1])
```

## Rule 4: Interference

- ▶ Quantum amplitudes can add (constructive) or cancel (destructive)
- ▶ This is similar to wave interference in optics and sound
- ▶ Quantum algorithms use interference to amplify the probability of the right answer
- ▶ Constructive interference: two paths to the correct answer add up
- ▶ Destructive interference: paths to wrong answers cancel out
- ▶ Interference is what makes quantum algorithms faster than brute force

## Rule 5: No-Cloning Theorem

- ▶ It is impossible to make a perfect copy of an unknown quantum state
- ▶ Classical bits can be copied:  $0 \rightarrow 00$ ,  $1 \rightarrow 11$
- ▶ Qubits cannot: copying would violate the laws of quantum mechanics
- ▶ This is not a technology limitation — it is provably impossible
- ▶ **Why it matters for computing:** You cannot “backup” quantum data mid-algorithm
- ▶ **Why it matters for security:** An eavesdropper cannot clone quantum encryption keys
- ▶ This is the foundation of quantum cryptography



## Rule 6: Decoherence

- ▶ Real qubits interact with their environment (temperature, electromagnetic noise)
- ▶ These interactions cause qubits to lose their quantum properties over time
- ▶ Decoherence: the qubit behaves increasingly like a classical bit with noise
- ▶ T1 time: how long a qubit stays in excited state  $|1\rangle$  (energy relaxation)
- ▶ T2 time: how long phase information survives (dephasing)
- ▶ Typical IBM superconducting qubits: T1, T2 in microseconds to milliseconds
- ▶ This limits how deep (long) our circuits can be

# Schrödinger's Cat: A Famous Thought Experiment

- ▶ A cat is in a sealed box with a quantum device that may or may not release poison
- ▶ Before opening: the cat is in a superposition of alive AND dead
- ▶ After opening (measurement): the cat is definitely one or the other
- ▶ Intended to highlight the absurdity of applying quantum rules to macroscopic objects
- ▶ Reality: large objects decohere almost instantly — superpositions don't survive
- ▶ Quantum superpositions only survive at the scale of atoms, electrons, photons

## Bell's Theorem: Quantum vs Classical Correlations

- ▶ Bell (1964) devised a test to distinguish quantum correlations from classical ones
- ▶ Classical correlations (hidden variable theory): limited by Bell inequalities
- ▶ Quantum entanglement violates Bell inequalities
- ▶ Experiments by Aspect (1982), Zeilinger (2022 Nobel) confirmed quantum violation
- ▶ Conclusion: quantum mechanics is fundamentally different from classical probability
- ▶ Practical consequence: entanglement enables protocols impossible classically (teleportation, QKD)

## Quantum Tunneling (Bonus: Not in Circuits, but Everywhere)

- ▶ A quantum particle can pass through a barrier it classically couldn't cross
- ▶ Analogy: a ball rolling up a hill, but it sometimes “teleports” to the other side
- ▶ Enables: nuclear fusion in stars, scanning tunneling microscopes, flash memory
- ▶ In quantum computing: used in quantum annealing (D-Wave) for optimization
- ▶ Not directly used in gate-model quantum computing (our course focus)
- ▶ Shows how quantum effects underlie everyday technology

## Quantum vs Classical: Side-by-Side Comparison

- ▶ **State:** Classical = definite, Quantum = superposition until measured
- ▶ **Copying:** Classical = free, Quantum = forbidden (no-cloning)
- ▶ **Measurement:** Classical = passive reading, Quantum = disturbs and collapses
- ▶ **Correlation:** Classical = can be explained locally, Quantum = entanglement is non-local
- ▶ **Interference:** Classical = none, Quantum = amplitudes add/cancel
- ▶ **Error:** Classical = bit flip is detectable, Quantum = also phase flip errors

## Lab: Entanglement and Measurement Correlation

- ▶ Create a Bell state circuit in Qiskit, run 1000 shots on simulator
- ▶ Observe: only 00 and 11 outcomes — never 01 or 10
- ▶ Extension: add a noise model to AerSimulator and observe correlation degradation

```
from qiskit_aer import AerSimulator
2 from qiskit.visualization import plot_histogram
  sim = AerSimulator()
4 result = sim.run(qc, shots=1000).result()
  plot_histogram(result.get_counts())
```

## Lab: Interference Demonstration

- ▶ Apply H twice:  $H|0\rangle = |+\rangle$ , then  $H|+\rangle = |0\rangle$  — amplitudes cancel back
- ▶  $|+\rangle$  and  $|-\rangle$  look identical in Z measurements, but interfere very differently
- ▶ Result: circuit always gives 0 — constructive then destructive interference at work

```
1 qc = QuantumCircuit(1, 1)
  qc.h(0)          # Create superposition: |+>
3 qc.h(0)          # Interfere back to |0>
  qc.measure(0, 0) # Always measures 0
```

## Exercise: Quantum Rule Identification

- Identify which quantum rule is at play in each scenario:
  1. A qubit is in state  $|+\rangle$ ; after measurement it always gives 0 in future — (Collapse)
  2. Two qubits always agree when measured, even when far apart — (Entanglement)
  3. An algorithm boosts probability of the correct answer using phase tricks — (Interference)
  4. After 100 microseconds, a qubit's quantum state degrades — (Decoherence)
  5. You cannot duplicate an unknown qubit to back it up — (No-Cloning)



## Module 2 Summary

- ▶ **Superposition:** qubits can be in multiple states simultaneously
- ▶ **Measurement:** collapses superposition; irreversible
- ▶ **Entanglement:** non-local correlations between qubits; experimentally confirmed
- ▶ **Interference:** amplitudes add/cancel; enables algorithmic speedup
- ▶ **No-cloning:** unknown quantum states cannot be copied; basis of quantum crypto
- ▶ **Decoherence:** quantum properties decay due to environmental interaction; limits circuit depth
- ▶ Next: how to control qubits with quantum gates and build circuits

# Quantum Machine Learning

# QML vs Traditional Neural Networks

- ▶ Similarities:
  - ▶ using “stacked layers” of transformations that make up a larger model
  - ▶ data is used to inform updates to model parameters, typically to minimize some loss function
- ▶ Difference: QML models have access to the power of quantum mechanics and deep neural networks do not.
- ▶ Variational quantum circuits (QVC) is the QNN, has encoder circuit, the variational circuit and the measurement operators.

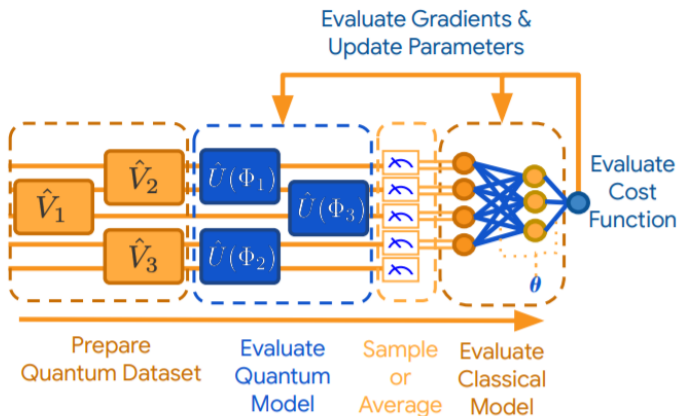
(Ref: My experience with TensorFlow Quantum, Owen Lockwood, Rensselaer Polytechnic Institute)

# QVC: Structure and Components

- ▶ The encoder circuit either takes naturally quantum data (i.e. a non-parametrized quantum circuit) or converts classical data into quantum data.
- ▶ Variational circuit is defined by its learnable parameters. The parametrized part of the circuit is the part that is updated during the learning process.
- ▶ Measurement operators extract information from the QVC some sort of quantum measurement (such as a Pauli X, Y, or Z basis measurements), a loss function (and gradients) is calculated on a classical computer and the parameters can be updated.
- ▶ QML can harness quantum phenomena, such as superposition and entanglement.

(Ref: My experience with TensorFlow Quantum, Owen Lockwood, Rensselaer Polytechnic Institute)

# QVC: Architecture Overview



(Ref: My experience with TensorFlow Quantum, Owen Lockwood, Rensselaer Polytechnic Institute)

# QVC: Quantum Phenomena in ML

- ▶ Superposition stems from the wave function being a linear combination of multiple states and enables a qubit to represent two different states simultaneously (in a probabilistic manner). The ability to operate on these superpositions, i.e. operate on multiple states simultaneously, is integral to the power of quantum computing.
- ▶ Entanglement is a complex phenomenon that is induced via multi-qubit gates.
- ▶ Near term and current quantum devices have 10s-100s of quantum bits (qubits) like the Google sycamore processor. Because of their size and the noise, this hardware is often called Noisy Intermediate Scale Quantum (NISQ) technology.
- ▶ For classical ML researchers with experience in TensorFlow, TFQ makes it easy to transition and experiment with QML at small or large scales. The API of TFQ and the modules it provides (i.e. Keras-esque layers and differentiators) share design principles with TF and their similarities make for an easier programming transition.
- ▶ **PennyLane** (Xanadu) is now the widely-used QML framework: hardware-agnostic, works with Qiskit, Cirq, and many backends; recommended for new QML projects.

(Ref: My experience with TensorFlow Quantum, Owen Lockwood, Rensselaer Polytechnic Institute)

## References

Many publicly available resources have been referred to for making this presentation. Some notable ones:

- ▶ Quantum Computing Lecture Series — Bhaskaracharya Pratishtan, 2021
- ▶ Berkeley course “Quantum Mechanics and Quantum Computation” — Prof. Umesh Vazirani
- ▶ Quantum Mechanics for Everyone — Georgetown-Edx course
- ▶ QPiA courses
- ▶ Nielsen & Chuang, *Quantum Computation and Quantum Information* (Cambridge, 2010)
- ▶ Udemy Course: <https://www.udemy.com/course/quantum-computers/learn/>
- ▶ Coursera: <https://www.coursera.org/specializations/quantum-computing-from-basics-to-the-cutting-edge>
- ▶ Qiskit Textbook: <https://qiskit.org/learn>
- ▶ PennyLane QML Demos: <https://pennylane.ai/qml/demonstrations/>
- ▶ Fematika (category theory) playlist: <https://www.youtube.com/c/Fematikaplaylists>
- ▶ Sabine Hossenfelder (Physics) playlist: <https://www.youtube.com/playlist?list=PLwgQsqstH9H5djIfFhXE6We207beTgUnyL>

# Thanks ...

- ▶ Office Hours: Saturdays, 3 to 5 pm (IST);  
Free – Open to all; email for appointment  
to yogeshkulkarni at yahoo dot com
- ▶ Call + 9 1 9 8 9 0 2 5 1 4 0 6



(<https://www.linkedin.com/in/yogeshkulkarni/>)



(<https://medium.com/@yogeshharibhaukulkarni>)



(<https://www.github.com/yogeshhk/>)